

Name/NetID: Nathan Kariuki / nkari011

Session Date: April 30, 2026 (2-3 hours)

Objective: Implement all 4 CC push variants.

Attempts made:

CC push is different because it treats edges as undirected. My first attempt only scanned out-edges which meant labels could only propagate forward and components never fully merged. The CC dense worklist frontier also needs to start with all vertices instead of just the source since every vertex starts as its own component.

Solutions/Workarounds:

CC Process scans both out-edges and in-edges:

```
for (uint32_t i = g.offsets[u]; i < g.offsets[u+1]; i++) {
    uint32_t v = g.edges[i];
    pool.lock(v);
    if (prev[u] < comp[v]) { comp[v] = prev[u]; in_next[v] = 1; updated[tid] = 1; }
    pool.unlock(v);
}
for (uint32_t i = g.in_offsets[u]; i < g.in_offsets[u+1]; i++) {
    uint32_t v = g.in_edges[i];
    pool.lock(v);
    if (prev[u] < comp[v]) { comp[v] = prev[u]; in_next[v] = 1; updated[tid] = 1; }
    pool.unlock(v);
}
```

Commits: Add CC serial naive push; Add CC serial dense worklist; Add CC parallel naive push with locks; Add CC parallel dense worklist with locks; Verify all CC variants match each other and Phase 2a output

Experiments/Analysis:

Tested on disconnected graph (0 1 1 / 1 2 1 / 3 4 1):

vertex 0 = 0

vertex 1 = 0

vertex 2 = 0

vertex 3 = 3

vertex 4 = 3

CC dense serial vs naive serial passed

CC naive parallel vs naive serial passed

CC dense parallel vs naive serial passed
CC naive serial vs Phase 2a passed

Learning outcome:

CC needs both edge directions to correctly merge components. Starting the frontier with all vertices is critical, if you start with just one vertex nothing propagates to the rest of the graph.